

Chapter 1: Kafka Recap

① message $\rightarrow$ (key, value)

(hash (key) % no of partitions)
deciding which partition should the message go.

This partition is present at the producer side.

② Batching of messages is done, to avoid so many huge load of messages to avoid so many network calls.

[Compression is applied as well].

③ Kafka itself stores chunked data. The producer does not store (key, value).

At producer side serialization is time and at consumer side deserialization is time.

JSON $\rightarrow$ JavaScript Object Notation.

XML $\rightarrow$ Extensible Markup Language.

These 2 formats are not version compatible.

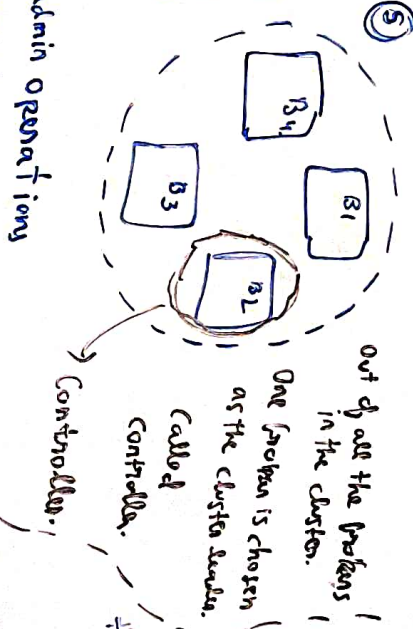
Avro is used here providing both forward & backward compatibility.

(Avro 2.0 & Avro 1.0)

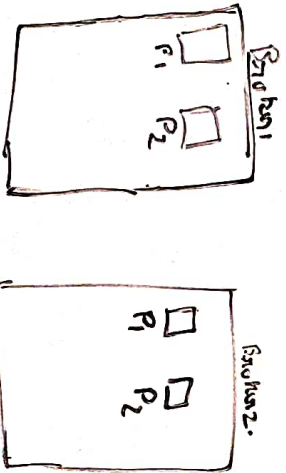
if any field is missing it will be null.

④ By default Kafka will produce messages evenly across all partitions, but we write logic to (hash (key) % no of partitions).

Producer produces message to a broker, Kafka automatically adds an offset metadata to it. (offset mentioned in increasing).



* admin operations
* assigning partitions to brokers.



P1's leader is Broker 1

Broker 2 is ISR for P1.

P2's leader is Broker 2

Broker 1 is ISR for P2.

⑤ Producer produces message to only one partition. Leader for that partition is the broker to which the producer connects.

Consumer on the other hand can connect to any follower node.

topic level Retention $\rightarrow$ 7 days.

Size level retention $\rightarrow$ 1 GB [Happening at partition level]

Log Compacted $\rightarrow$ Another new kind of retention that can be used.

For a given key only retain the last message for that key.

offset	key	value
1	101	
2	102	
4	101	
5	102	

only those two are retained basically giving latest offset.

Used in CDC, to figure out latest change log, in DB.

⑥ Minor Replication used to replicate across multiple clusters.